

Marn — Flows

Call chains: what calls what, across which files/functions/modules, in what order

Marn — companion app for physical rehab and pain relief

Execution flow log

Traces how an action moves through the code — which file/function/module calls which, in what order — for the paths that have actually been touched under this process. Complements `docs/architecture/overview.md` (module ownership and phase status) and `docs/decisions.md` (why a choice was made); this file is strictly "what calls what."

Not a full trace of the existing codebase up front — populated incrementally, one flow per change, as a change touches that path. A flow entry is **updated in place** (not duplicated) the next time a change touches the same path, so this stays a current map, not a chronological diary.

Entry format

```
## <Flow name — e.g. "Member PAR-Q submission">
**Entry point:** <route / component / CLI command>
**Path:**
1. `File.tsx` `ComponentName` (file:line) — <what it does> →
2. `file.ts` `functionName()` (file:line) — <what it does> →
3. ...
**Currently modifying:** <step number(s) and what the active change does
there — remove this line once the change lands; the rest becomes the static
record>
```

Root app boot — fonts, theme, providers

Entry point: any request to the root Next.js app (`app/`) **Path:**

1. `app/layout.tsx` `RootLayout()` (`app/layout.tsx:29`) — loads Petrona/Figtree via `next/font/google`, exposes them as CSS variables (`--font-petrona`, `--font-figtree`) on `<html className>` →
2. `app/layout.tsx` — renders `InitColorSchemeScript` (inline, pre-hydration) to stamp `data-mui-color-scheme` on `<html>` before paint, avoiding a light/dark flash →
3. `theme/theme.ts` `theme` (`theme/theme.ts:43`) — `createTheme()` with `cssVariables` + `colorSchemes.dark` / `.light`, reading the two font CSS variables into `typography*.fontFamily` →
4. `app/layout.tsx` — `AppRouterCacheProvider` → `ThemeProvider theme={theme}` → `CssBaseline` (applies the `MuiCssBaseline` override: `tabular-nums`, focus ring) → `{children}`

Staff sign-in and console routing

Entry point: `/login` **Path:**

1. `app/login/page.tsx` `LoginPage()` `onSubmit` (`app/login/page.tsx:26`) — client form calls `signIn.email()` →
2. `lib/auth-client.ts` `signIn` (`lib/auth-client.ts:8`) — better-auth react client, POSTs to `/api/auth/sign-in/email` →

3. `app/api/auth/[...all]/route.ts` `POST` — `toNextJsHandler(auth)` from `lib/auth.ts` 's `betterAuth()` config, sets the session cookie →
4. on success, client `router.push('/')` →
5. `app/page.tsx` `Home()` (`app/page.tsx:6`) — `getStaffSession()` (`lib/authz.ts:21`) resolves the cookie to a staff row, then `redirect()` s to `/coach` or `/studio` by role (or `/login` if no session)

`app/coach/page.tsx`, `app/studio/page.tsx`, and `app/superadmin/page.tsx` all repeat the same `getStaffSession()` + role-mismatch redirect at the top of the route, not just here — this is the front door, not the only lock (every `lib/actions/*.ts` function re-authorizes independently underneath it). The mismatch redirect target is `roleHome(session.role)` (`lib/authz.ts`), not a hardcoded sibling route — a superadmin bouncing off `/studio` lands on `/superadmin`, not `/coach`.

Studio manager: manual booking creation, with overlap protection

Entry point: `/studio` Floor tab, "New booking" form **Path:** 0. `components/studio/TimeSlotPicker.tsx` — once member/coach/service/date are all picked, fetches `getCoachDayAvailability()` (`lib/actions/bookings.ts`) and renders one chip per candidate start time, disabling any that fall outside the coach's assigned shift or overlap an existing booking (`lib/scheduling.ts` 's `computeFreeSlots`) — a manager can only ever *select* a slot the write path would also accept →

1. `components/studio/FloorPanel.tsx` `onSubmit` (`components/studio/FloorPanel.tsx`) — client form collects member/coach/service/date/time, calls `createBooking` →
2. `lib/actions/bookings.ts` `createBooking()` — `requireStudioManager()` gate, opens `db.transaction()` →
3. `lib/actions/bookings.ts` `assertNoOverlap()` (same file) — `lib/scheduling.ts` 's `bookingRange()` computes the candidate `[start,end]`, rejects outside `STUDIO_HOURS`; queries same-day bookings sharing either the coach or the member; `computeBusyRanges()` + `rangesOverlap()` check both, throwing a plain `Error` (surfaced verbatim to the UI by `components/studio/primitives.tsx` 's `useFormSubmit`) on either conflict →
4. on no conflict, the same transaction inserts the `bookings` row as `confirmed` →
5. after the transaction commits, `logAudit(..., 'booking_approved', ...)` (`lib/audit.ts`) — outside the transaction, matching the pre-existing pattern (audit logging is best-effort, not part of the write's atomicity guarantee).

`rescheduleBooking()` and `reassignCoach()` (same file) share step 3's `assertNoOverlap()` via `loadActiveBooking()` (loads + site-scopes + status-gates the target booking first, `excludeBookingId` lets the booking's own current slot stay "free" during the check), wired to `components/studio/FloorPanel.tsx` 's `RescheduleAction` / `ReassignAction` row-level `Popover` s (module-scope, share the panel's single `useFormSubmit` `run` so success/error surface through the one Alert this panel already has) — each success callback also calls `refreshDay()` (`getDaySchedule(viewDate)`) so the floor table and `components/studio/DayTimeline.tsx` update without a page reload.

Studio manager: member detail drawer

Entry point: `/studio` Members tab, "View details" row action **Path:**

1. `components/studio/MembersPanel.tsx` — `setSelected(member)` on click, renders `MemberDetailDrawer` with that member →

2. `components/studio/MemberDetailDrawer.tsx` — on open, `Promise.all([getMemberContext(member.id), getMemberBookingHistory(member.id)])` →
3. `lib/actions/members.ts` `getMemberContext()` — `requireStaff()` + `assertMemberInScope()` (full-detail pass-through for `studio_manager`), returns identity/checkins/sessions/assessments/measurements/flags — same function `components/coach/MemberContextPanel.tsx` calls, role-branched identity →
4. `lib/actions/bookings.ts` `getMemberBookingHistory()` — `requireStudioManager()` + `assertMemberInScope()`, site-scoped booking rows, newest first →
5. drawer renders four read-only-or-link-management sections: `MemberAccessSection` (module-scope, fetches `getActiveMemberAccessToken()` on open — see the member-portal-access flow below), Measurements, Session history, Booking and charge history.

Member portal: staff issues a link, member views it

Entry point: `/studio` Members tab → detail drawer's "Their progress link" (generation); `/m/[token]` (member's own view)

Path (staff issues/regenerates a link):

1. `components/studio/MemberDetailDrawer.tsx` `MemberAccessSection` — "Generate link" → `generateMemberAccessLink(memberId)` (`lib/actions/memberAccess.ts`) →
2. `requireStaff()` + `assertMemberInScope()`, revokes any existing non-revoked `member_access_tokens` row for that member, inserts a fresh one (`crypto.randomBytes(24)`, `base64url`), `logAudit(..., 'member_access_link_created', 'member', memberId)` →
3. component builds `${window.location.origin}/m/${token}` client-side and copies it via `navigator.clipboard`.

Path (member opens the link):

1. `app/m/[token]/page.tsx` — no session/auth check, calls `getMemberPortalData(token)` (`lib/actions/memberPortal.ts`) →
2. looks up the token in `member_access_tokens` (must be non-revoked), resolves the owning `members` row; a missing/revoked token short-circuits to `null` →
3. `lib/scores.ts` `getMemberScores()` — loads the latest `assessments` / `measurements`, the most recent `sessions.rpe`, calls `lib/scoring.ts`'s `computeScores()` / `priorityAreas()` (`adherence` / `streak` / `hasWearable` all placeholder-zero, no home-programme data source yet) →
4. page also loads the 20 most recent `sessions` rows, then renders `components/member/MemberPortal.tsx` (scores, priority areas, region-grouped range list, session history) — or the "link isn't valid" state if step 2 returned `null`.

Superadmin: acting as another staff member ("View as")

Entry point: the "View as" dropdown in the AppBar, on any console **Path:**

1. `components/StaffChrome.tsx` renders `ImpersonationSwitcher` only when its `impersonation` prop is non-null — set by `getImpersonationContext()` (`lib/actions/impersonation.ts`), which returns `null` for anyone whose **real** session isn't an active superadmin. All three pages (`app/{coach,studio,superadmin}/page.tsx`) fetch it alongside their own reads →

2. selecting a name → `startImpersonation(staffId)` — authorizes via `getRealStaffSession()` (never `getStaffSession()`, which would report the borrowed role), validates the target is an active non-superadmin, sets the httpOnly `marn_act_as` cookie, `logAudit(realSuperadminId, 'staff_impersonated', ...)` →
3. client `router.push('/')` → `app/page.tsx` → `roleHome(session.role)` routes to whichever console the *effective* role now owns →
4. every subsequent request: `lib/authz.ts` `getStaffSession()` → `getRealStaffSession()` first; if that's an active superadmin **and** the cookie is present **and** the target row is still active and non-superadmin, returns the target's session with `impersonatedBy` set; otherwise falls back to the real session. Every existing `require*` /action underneath is unchanged and simply sees the effective identity →
5. `ImpersonationBanner` (same file as the switcher) renders under the AppBar whenever `impersonatedBy` is set; its "Back to superadmin" calls `stopImpersonation()` → cookie deleted → step 3's routing runs again, landing back on `/superadmin`.

Superadmin: overview dashboard, coach workload, cash ledger

Entry point: `/superadmin` (Overview tab, default) **Path:**

1. `app/superadmin/page.tsx` — session/role gate (`getStaffSession` + `roleHome`), then `Promise.all([getSuperadminDashboard(), getCoachWorkload(), getCashLedger()])` →
2. `lib/actions/dashboard.ts` `getSuperadminDashboard()` — `requireSuperadmin()`, 4 unscoped/near-unscoped queries (`sites`, today's bookings, 7d bookings, active staff), grouped per site in JS, plus a reduced platform total →
3. `lib/actions/coachWorkload.ts` `getCoachWorkload()` — `requireStudioManagerOrSuperadmin()` (shared with the studio console's future use of the same action), no `siteId` arg here so every site's coaches are included; shift/booking/session queries scoped to the coach id list, hours computed via `lib/scheduling.ts` `'s` `timeToMinutes` →
4. `lib/actions/cashLedger.ts` `getCashLedger()` — `requireSuperadmin()`, unions derived booking revenue (`bookings.aed` where `status='completed'`) with manual `cash_ledger` rows, sorted newest-first →
5. `components/superadmin/SuperadminConsole.tsx` → `OverviewPanel.tsx` renders all three; the ledger panel's "Record a cash entry" form calls `recordCashEntry()` (same file as `getCashLedger`), which `router.refresh()` s on success to pull the new entry back through step 4.

Superadmin: create a studio, create/reassign staff to it

Entry point: `/superadmin` Studios & staff tab **Path (create site):**

1. `components/superadmin/StudiosPanel.tsx` `onCreateSite` → `createSite()` (`lib/actions/staff.ts`) — `requireSuperadmin()`, inserts a `sites` row, `logAudit(..., 'site_created', ...)`.

Path (create staff at a site):

1. Same panel's second form → `createStaffAccountForSite()` — `requireSuperadmin()`, `auth.api.signUpEmail()` then inserts a `staff` row with the caller-chosen `siteId` / `role` (`'coach'` | `'studio_manager'` only — never `'superadmin'`, see `docs/decisions.md`).

Path (reassign existing staff):

1. `StudiosPanel.tsx` 's `ReassignAction` (module-scope, same Popover pattern as `components/studio/FloorPanel.tsx`) → `assignStaffToSite()` — `requireSuperadmin()`, updates `staff.siteId`, `logAudit(..., 'staff_site_assigned', ...)`.

All three call `router.refresh()` on success (via the shared `useFormSubmit`), which re-runs

`app/superadmin/page.tsx` 's `getSites()` / `getAllStaff()` server-side and flows fresh props back down — same mechanism as every other write in either console.

Coach: member context load and in-place capture

Entry point: `/coach`, selecting a member from the schedule or roster **Path:**

1. `app/coach/page.tsx` `CoachPage()` (`app/coach/page.tsx:12`) — `getStaffSession()` + role gate, then `getCoachScheduleToday()` / `getCoachMembers()` in parallel, narrowed field-by-field before reaching the client →
2. `components/coach/CoachConsole.tsx` `onSelect()` (`components/coach/CoachConsole.tsx:183`) → `load()` (`components/coach/CoachConsole.tsx:170`) →
3. `lib/actions/members.ts` `getMemberContext()` (`lib/actions/members.ts:68`) — re-authorizes via `requireStaff()`, then `assertMemberInScope()` (`lib/authz.ts:76`) before touching any per-member table; also loads the latest `parq_screenings` row for the referral-note banner →
4. `components/coach/MemberContextPanel.tsx` renders the result; `ParqScreeningForm` / `MeasurementCapture` / `SessionLogForm` (`components/coach/CaptureForms.tsx`) call `submitParqScreening` / `createManualAssessment` / `logSession` on save →
5. save calls `onChangeed()` back in `CoachConsole.tsx`, which re-runs step 2–3 (stale-while-revalidate — the panel never unmounts) and `router.refresh()` s the server-fetched schedule/roster

Coach: readiness screening (PAR-Q)

Entry point: `/coach`, member context panel, "Start screening"/"Re-screen"

Path: 0. `lib/actions/members.ts` `getCoachMembers()` unions the coach's normally-assigned members with *every not-yet-screened member at the site* — without this branch a first-time member (no booking/session yet, since `createBooking` now refuses unscreened members) would never appear in any coach's roster at all, a deadlock caught via browser-driven verification (`docs/decisions.md`, 2026-08-11) →

1. `components/coach/CaptureForms.tsx` `ParqScreeningForm` (module-scope, per the known-trap rule — survives the panel's background refresh with answers intact) — checkbox per `lib/reference.ts` 's `PARQ_QUESTIONS`, optional note, calls `submitParqScreening(memberId, answers, note)` →
2. `lib/actions/parq.ts` `submitParqScreening()` — `requireCoach()` + `assertMemberInScope()` (also widened: an unscreened member is in scope for any coach at the site, not just an assigned one — same fix, `lib/authz.ts`); `redFlag` is `true` if any answer marked `redFlag: true` in `PARQ_QUESTIONS` was checked →
3. inserts a `parq_screenings` row (full answers, `redFlag`, note, staff/site attribution), then updates `members.parqCleared` / `parqAt` — `!redFlag` / `now()` if clean, `false` / `null` if red-flagged (a fresh red flag always overwrites a prior clearance) →
4. `logAudit(..., 'readiness_changed', 'member', memberId)` →
5. `lib/actions/bookings.ts` `createBooking()` reads `members.parqCleared` (via `assertMemberInScope` 's return) before `assertNoOverlap` — an uncleared member's booking is refused with a plain error, no separate flow.

Once cleared, step 0's widened branch no longer applies and the member reverts to ordinary booking/session-tied scoping (confirmed live: the coach's roster dropped the member again immediately after a clean screening, before any booking existed).

Studio manager: manual booking intake

Entry point: `/studio`, "New booking" form on the Floor tab **Path:**

1. `app/studio/page.tsx` `StudioPage()` (`app/studio/page.tsx:18`) — `getStaffSession()` + role gate, then five reads in parallel including `getManagerDashboard()` (`lib/actions/dashboard.ts`) →
2. `components/studio/FloorPanel.tsx` `onSubmit()` (`components/studio/FloorPanel.tsx:57`) — price/duration read from `lib/reference.ts`'s `SERVICES`, never typed →
3. `lib/actions/bookings.ts` `createBooking()` (`lib/actions/bookings.ts:39`) — `requireStudioManager()`, `assertMemberInScope()` then refuses if `members.parqCleared` is false (readiness screening flow, above) before `assertNoOverlap`; inserts a `confirmed` booking with a coach already assigned (no separate request-then-approve step in this slice — `docs/decisions.md`, 2026-08-11), writes an audit log row →
4. `components/studio/primitives.tsx` `useFormSubmit()`'s `run()` shows the success `Snackbar` and calls `router.refresh()`, which re-runs step 1 server-side

Member: self-registration

Entry point: `/join`

Path:

1. `app/join/page.tsx` `JoinPage` `onSubmit` — client calls `authClient.signUp.email({ email, password, name })` (`lib/auth-client.ts`) directly, not a server action, so the browser receives the better-auth session cookie the normal way →
2. on `onSuccess`, still client-side, calls `completeMemberRegistration({ phone, siteId })` (`lib/actions/memberAuth.ts`) →
3. `completeMemberRegistration()` — `auth.api.getSession()` reads the now-live session from request headers, inserts a `members` row with `authUserId: session.user.id`, `addedByStaffId: null` (the self-registered counterpart to the studio manager's `createMember`) →
4. client `router.push('/member')` → `app/member/page.tsx` → `getMemberSession()` (`lib/memberAuth.ts`) resolves the same session to the new `members` row, `getMyPortalData()` (`lib/actions/memberSelf.ts`) loads the (empty, first-run) scores/sessions →
5. `components/member/MemberConsole.tsx` renders — Book tab shows the readiness-pending state (`session.parqCleared` is `false` for every new member) rather than a booking form, matching `docs/flow.md`'s "Coach: readiness screening" entry's step 0 — this new member is exactly who that unscreened-member-scoping widening exists for.

Member: self-service booking, through to studio manager approval

Entry point: `/member`, "Book" tab (only reachable in form once `parqCleared`)

Path:

1. `components/member/BookingForm.tsx` — service picker (`lib/reference.ts` 's `SERVICES` , price never typed), `getActiveCoachesAtSite()` (`lib/actions/bookings.ts` , `requireMember()` -gated, id/name only) for the coach picker, `components/shared/TimeSlotPicker.tsx` fed `getMemberAvailability` (same `computeCoachAvailability` `CORE` `getCoachDayAvailability` uses, just member-authorized and site-locked to the member's own `siteId`) →
2. submit → `createSelfBooking()` (`lib/actions/bookings.ts`) — `requireMember()` , refuses if `!session.parqCleared` (same message shape as `createBooking` 's), reuses `assertNoOverlap` inside the same transaction that also inserts a `credit_ledger` `consumption` entry (`-1` , unconditional — not balance-gated this pass, `docs/decisions.md` 2026-08-12), inserts with `status: 'requested'` (not auto-confirmed — see `createBooking` 's doc comment, which anticipated this exact path), then `notifyRecorded(..., 'booking_requested', ...)` outside the transaction →
3. `BookingForm` 's `onBooked` callback → `MemberConsole.tsx` 's `refreshBookings()` → `getMemberOwnBookings()` , "My bookings" tab shows it as "Awaiting confirmation" →
4. separately, `/studio` Floor tab's existing `requested / confirmed` row rendering (`components/studio/FloorPanel.tsx` , unchanged from Phase 1) now shows this row with an **Approve** button (new, only rendered when `status === 'requested'`) alongside the existing `Reschedule/Reassign/Decline` →
5. "Approve" → `approveBooking()` (`lib/actions/bookings.ts`) — `requireStudioManager()` , `declineBooking` 's sibling: sets `confirmed` , `approvedByStaffId` / `approvedAt` , `logAudit(..., 'booking_approved', ...)` (same action value `createBooking` already writes) →
6. member's next `getMemberOwnBookings()` read (tab revisit, or after `cancelSelfBooking` on a different row) shows "Confirmed." `cancelSelfBooking()` is the only other state transition a member can trigger directly — see the cancellation-policy flow below for what it does now.

Member: cancellation, 24h policy

Entry point: `/member` , "My bookings" tab, "Cancel" on an active row

Path:

1. `components/member/MyBookings.tsx` 's client-side `willRefund(date, time)` (mirrors the server's own check, display-only) shows "refunded" or "not refunded" in the confirm dialog *before* the member commits — same hours-until-appointment math on both sides, only the server's copy is authoritative →
2. confirm → `cancelSelfBooking()` (`lib/actions/bookings.ts`) — `requireMember()` , own booking only, `requested / confirmed` → `cancelled` ; computes `hoursUntil(booking.date, booking.time)` server-side; inside one transaction, $\geq 24h$ inserts a `credit_ledger` `refund` (+1, `relatedBookingId`), $< 24h$ inserts nothing (forfeited) →
3. `notifyRecorded(..., 'booking_cancelled', { refunded })` outside the transaction →
4. returns `{ refunded }` to the client, which is what step 1's dialog copy is confirming ahead of time, not guessing at →
5. separately, `declineBooking()` (studio manager, `/studio` Floor tab) does the same lookup-and-refund check unconditionally — a manager declining a self-booking always refunds, regardless of timing, since the member didn't cause it; it only writes a `refund` entry if a `consumption` entry exists for that `relatedBookingId` in the first place (a staff-created booking never wrote one, so declining one of those touches the ledger at all only for self-bookings).

Coach: PAR-Q clearance keeping the coach's own access (the second scoping bug)

Entry point: `/coach`, same member context panel already open after clearing PAR-Q

Path:

1. Coach clears a member via `submitParqScreening()` (existing flow, `docs/flow.md` 's "Coach: readiness screening" entry) — `members.parqCleared` flips to `true` →
2. `onSaved()` → `CoachConsole.tsx` 's `onChange()` → re-runs `getMemberContext(memberId)` for the *same* member, in the *same* session, to refresh the panel →
3. `assertMemberInScope()` (`lib/authz.ts`) — the member is no longer unscreened (step 1), and has no booking/session tie yet, so **without the fix below** this throws `ForbiddenError`, breaking the panel the coach is actively looking at →
4. the fix: a new check queries the member's most recent `parq_screenings` row; if `staffId` matches the calling coach, access is granted anyway. `getCoachMembers()` (`lib/actions/members.ts`) carries the matching roster-level fix (same "latest screener" lookup) so the member doesn't also vanish from the left-rail list mid-session →
5. once an ordinary booking or session exists for that coach+member pair, neither branch is needed anymore — the original booking/session check already covers it.

Coach: prescribe a home programme, member completes it

Entry point: `/coach`, member context panel, "Prescribe programme"; `/member`, "Programme" tab

Path:

1. `components/coach/CaptureForms.tsx` 's `PrescribeProgramForm` — one fixed template (`PROGRAM_TEMPLATE`, matches the existing single-template precedent for session programmes) → `prescribeProgram()` (`lib/actions/programs.ts`) — `requireCoach()` + `assertMemberInScope()` (same widened/latest-screener scoping as above), inserts a `programs` row, `logAudit(..., 'program_prescribed', ...)` →
2. member's `/member` "Programme" tab (`components/member/ProgramTab.tsx`) — `getMyProgram()` (`requireMember()`), shows the moves and a "Mark today complete" button, disabled if today's date is already in `completions` →
3. tap → `markProgramComplete()` (`lib/actions/programs.ts`) — `requireMember()`, own programme only; appends today's ISO date to `completions` only if not already present (idempotent per day, no unique constraint — app-level check, matches `checkins` ' own per-day pattern) →
4. next `getMemberScores()` read (`lib/scores.ts`) — `cadencePerWeek` derived from `program.moves.length` (judgment call, `docs/decisions.md`), `adherence` computed from completions in the last 28 days ÷ expected, feeding both `recoveryScore` and the now-real `consistencyScore()` (`lib/scoring.ts`) — visible on both the member's own Overview and the coach's "Current programme" section (`components/coach/MemberContextPanel1.tsx` 's `ProgramSection`, completions-in-28-days count only, no score shown to the coach).

Member: pre-session check-in, coach sees it before the visit

Entry point: `/member` , "Check-in" tab

Path:

1. `components/member/CheckinForm.tsx` — sleep/pain sliders, a region chip list (Lower/Core/Upper — not an interactive body diagram, same scope call as the member portal's body map), optional note → `submitCheckin()` (`lib/actions/checkins.ts`) — `requireMember()` , looks for an existing `checkins` row for that member `at >=` today's midnight; updates it if found, inserts if not (idempotent per day, no schema constraint) →
2. `components/coach/MemberContextPanel.tsx` 's existing `CheckinsSection` (built in Phase 1, previously always empty since nothing wrote to `checkins`) now has a real row to show — no new coach-side code needed, only the write path was missing.

Notification triggers → `notifications` table

Entry point: six existing action call sites, no new UI

Path:

1. `lib/integrations/notifications/index.ts` 's `notifyRecorded(input)` — the swappable port's only implemented function (`docs/adr/0015`), a plain `db.insert(schema.notifications)` →
2. called from, one line each, no shared wrapper:
`createSelfBooking` / `approveBooking` / `declineBooking` / `cancelSelfBooking` (`lib/actions/bookings.ts`),
`submitParqScreening` (`lib/actions/parq.ts`), `completeMemberRegistration` (`lib/actions/memberAuth.ts`) — each passes a `template` name and a `payload` containing only booking/schedule-shaped data, never PAR-Q answers, measurements, or flag content (the Iron Rule against health data in log-adjacent surfaces applies here by convention at each call site, not a schema constraint) →
3. nothing reads the table yet — verified directly via the database in this pass, not through a UI, since no inbox exists.

Mobile: sign-in through to a booking, via the new REST layer

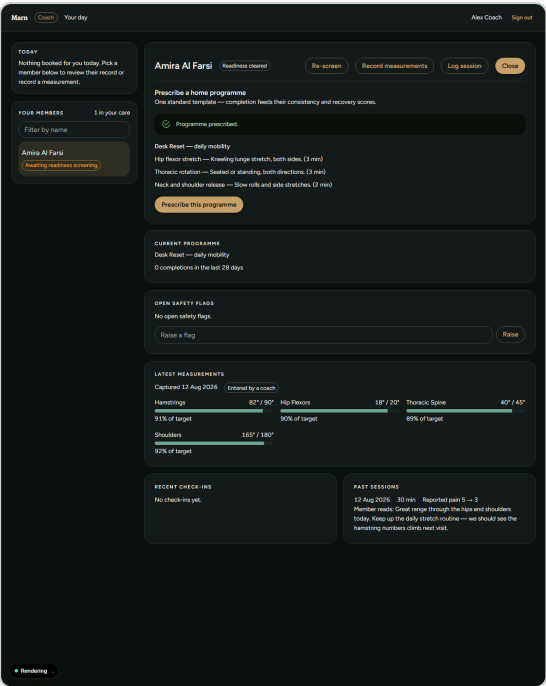
Entry point: `mobile/app/sign-in.tsx` (or `sign-up.tsx` , same two-step split as the web `/join` flow)

Path:

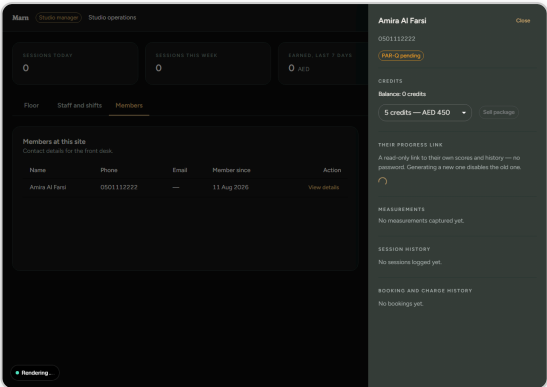
1. `mobile/lib/authClient.ts` 's `signIn.email` / `signUp.email` — the same better-auth instance as web (`docs/adr/0014`), now with `bearer()` and `expo()` plugins added (`lib/auth.ts` , `docs/adr/0017`); `expoClient()` stores the session in `expo-secure-store` →
2. every subsequent call goes through `mobile/lib/api.ts` 's `request()` — platform-branched (found necessary by direct reproduction, not assumed): native attaches the stored session via a manual `Cookie` header (`getCookie()`), web uses `credentials: 'include'` and the browser's own cookie jar, since browsers refuse to let script set a `Cookie` header at all →
3. `app/api/mobile/*/route.ts` (root Next.js app) — thin wrappers, e.g. `GET /api/mobile/portal` → `getMyPortalData()` , `POST /api/mobile/bookings` → `createSelfBooking()` — same functions the web member console calls, auth resolved from the request's cookie/bearer via `getMemberSession()` / `requireMember()` unchanged →

4. `middleware.ts` (new, root) adds CORS headers scoped to `/api/mobile/*` and `/api/auth/*` only — needed for the `expo start --web` verification target (a different origin than the Next.js dev server), not for native (no browser, no CORS) or for the existing staff/web-member routes (same-origin, untouched) →
5. `mobile/app/(app)/book.tsx` — first calls `GET /api/mobile/session` (new route, wraps `getMemberSession()`) to read `parqCleared / referredToDoctor` before showing the booking form at all, mirroring `components/member/BookingForm.tsx` 's own gate — a real gap the first verification pass caught (the mobile Book screen initially showed the form unconditionally) and this closes.

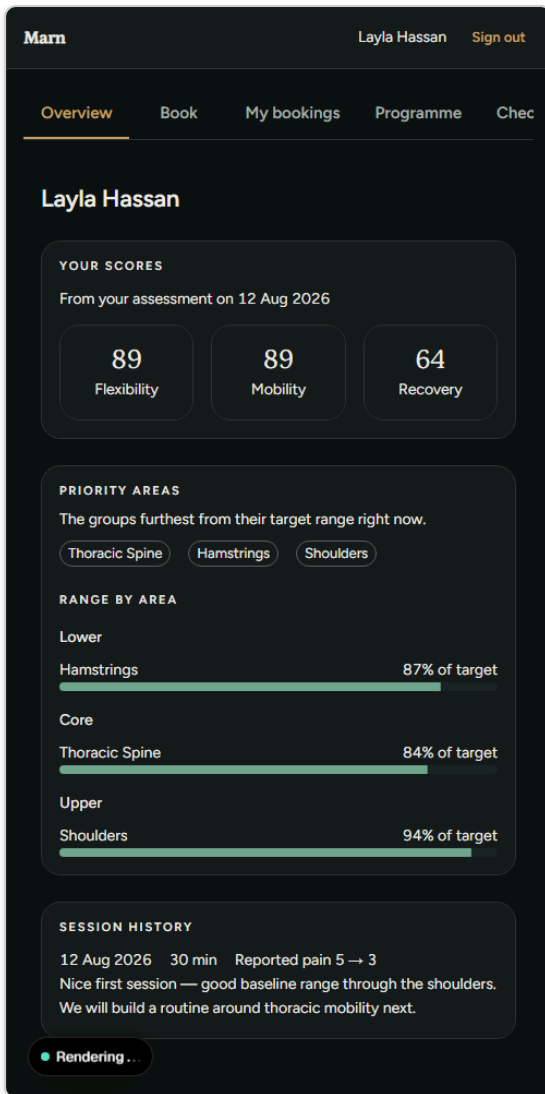
Appendix — Screens referenced in these flows



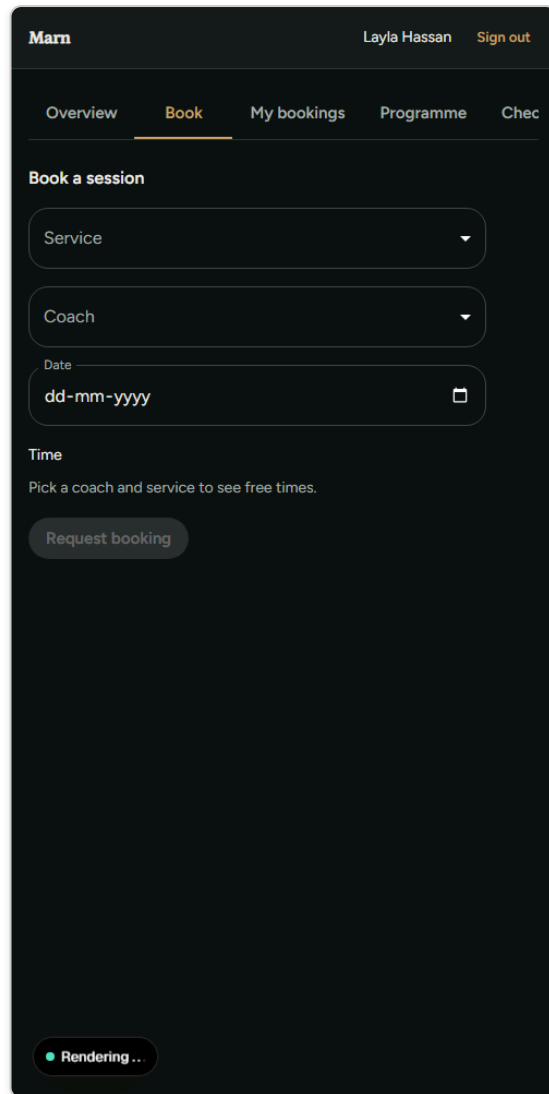
Coach console — PAR-Q screening, measurements, session log, programme prescription



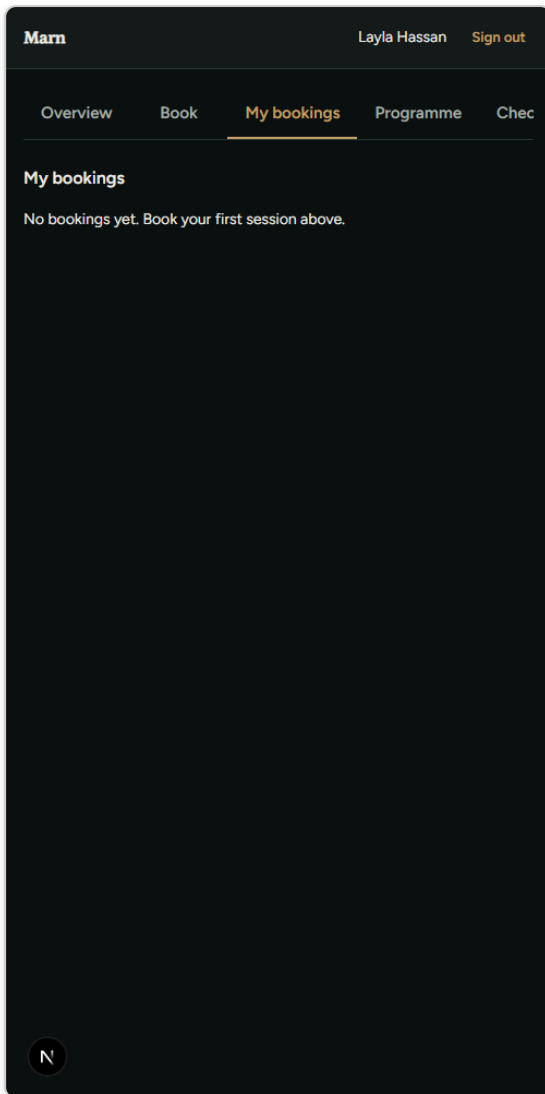
Studio manager — package purchase (credit ledger)



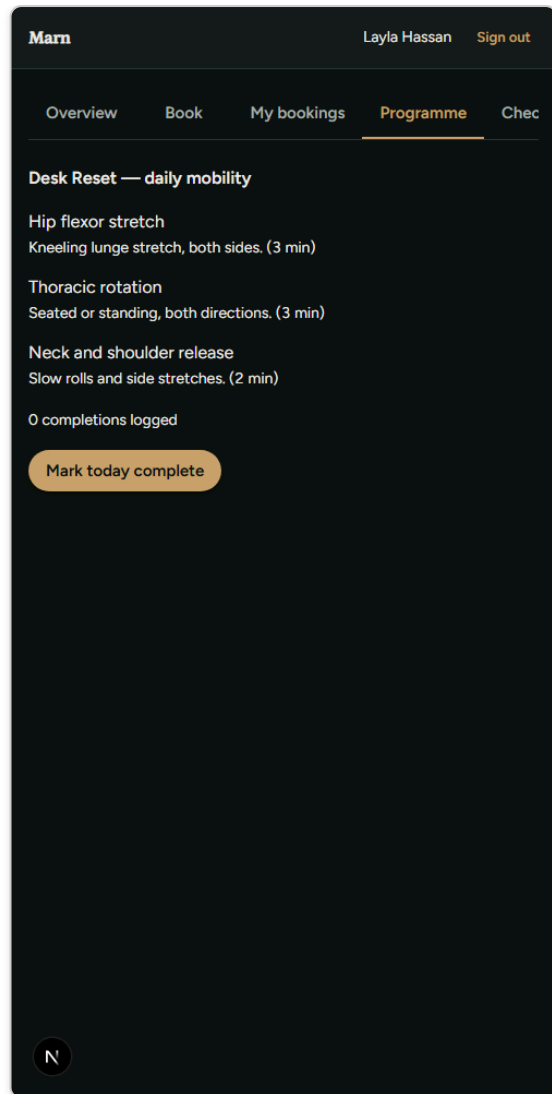
Member Overview after scoring



Member self-service booking



Member — cancellation flow entry point



Member — marking programme completion

Marn
Layla Hassan
Sign out

review
Book
My bookings
Programme
Check-in

Before you arrive

Two taps: how you slept, your current pain, and any areas bothering you.

Sleep quality

0 2 4 6 8 10

Current pain

0 2 4 6 8 10

Areas

Lower Core Upper

Note (optional)

Send check-in

Member — pre-session check-in

Overview

Layla Hassan

YOUR SCORES

89 89 64 0

Flexibility Mobility Recovery Consistency

SESSION HISTORY

2026-08-11T21:16:39.628Z · 30 min

Nice first session — good baseline range through the shoulders. We will build a routine around thoracic mobility next.

SIGN OUT

Overview Book My bookings Programme Check-in

Mobile — same data via the bearer-auth API layer

Book

Book a session

SERVICE

Assisted Stretch

Assisted Stretch — Long

Compression Recovery

Oxygen Reset

COACH

Alex Coach

DATE (YYYY-MM-DD)

2026-08-15

TIME

REQUEST BOOKING

Overview

Book

My bookings

Programme

Check-in

Mobile — booking flow

Marn — Page 16 of 16